

Generating Runtime-Verification Specs from Natural Language

Daniel Feng

April 29, 2026

1 Abstract

Runtime Verification (RV) theory, or the theory of analyzing how a software system changes operates over time, involves continuously monitoring execution traces of software to determine if the software satisfies requirements set by humans [7, 2]. However, turning natural language (NL) specifications such as "opening a file is will eventually be followed by closing the file" into formal logic is time-consuming and creates a barrier of entry for use of RV tools (in operations such as debugging). We present an transformer-based translation pipeline that helps novice developers leverage RV. The core of the architecture we propose is 3 transformers: an NL Clarifier, an Event Interpreter, and a Translator. Further, we create a novel ERE comparison tool in order to expand testing to larger datasets (in this case, a dataset of Java API RV specs). We received an accuracy of 61% on the dataset provided by Legunsun [8], moving a significant step forwards in making RV more accessible and faster.

Keywords: *Runtime-verification, MOP, JavaMOP, Large Language Models, Natural Language, Translation, Prompt Engineering, Fine-tuning, Java API*

2 Introduction

2.1 Context

The scale at which software is now being developed, spanning not only across multiple files but distributed systems, makes it infeasible to perform step-by-step debugging by hand. One solution is unit testing, but it is difficult to catch all cases. Thus, many developers choose to use Runtime Verification (RV). RV allows developers to create logic, and then as the software systems progresses temporally, a RV tool will ensure the logic is satisfied. For example, developers may write logic specs that say "Every file opened must be closed" or "Never modify a collection when it is being accessed."

2.2 Problem Statement

Of course, RV tools do not understand the natural language (NL) specs. Instead, these specs must be written in something called formal logic (rigorous mathematical equations and code that explicitly define the boundaries of what is and is not allowed).

However, writing this formal logic is difficult. It requires learning the runtime-verification framework being operated under, as well as having a deep understanding in the language of formal logic. Thus, in this paper, we attempt to automate the generation of formal logic.

2.3 Objective & Scope

We will attempt to use transformers (focusing on one kind of transformer, Large Language Models (LLMs)) to power the automated translation pipeline. We operate under the Monitoring-Oriented Programming (MOP) framework, which utilizes MOP files. Each MOP file consists of 4 main parts: Parameterization, Events, Logic, and Handlers [2, 7]. The Logic is the core of the MOP file, which combines all the other parts. There are multiple language plugins possible for use in MOP, but we focus on Extended Regular Expressions (ERE).

3 Background

3.1 Monitoring-Oriented Programming

In this section we clarify more on how MOP works. The Parameterization defines when the monitor begins tracking. The Events compartmentalize raw execution trace into well defined functions. The Logic uses those Events to create a specification that can either "match" or "fail." Finally, the Handler decides what happens when the specification matches and what happens when it fails [2].

3.1.1 Extended Regular Expressions (ERE)

We will write in ERE for this project. To summarize, ERE adapts the idea from Regular Expression (RE) and expands it onto events. While RE matches a pattern of characters onto a string, ERE matches a pattern of events onto the execution trace. Most of the same operations exist: the Kleene closure (*), the positive star closure (+), alternation (|), etc. Importantly, the optional quantifier (?) does not exist. Further, ERE allows the definition of creation events, which means that the monitor does not start tracking until this first event occurs. In terms of RE, this can be paralleled to having both a matching RE and a creation RE, where the matching RE does not start tracking until the creation RE matches. Further, ERE automatically stops tracking terminating events. In terms of RE this is equivalent to if the string was growing over time but the first time point where the RE matches it stops trying to match.

3.1.2 Linear Temporal Logic (LTL)

Linear Temporal Logic (LTL) evaluates the state of a system over continuous, infinite paths of time using specific temporal modalities. It combines standard propositional logic operators like AND ($\wedge$) and implication ($\rightarrow$) with time-specific operators—such as "Globally" (G), "Next" (X), and "Until" (U)—to construct complex, time-dependent rules about future system states.

3.2 Evaluation of Logic

We will analyze both syntactic and semantic similarity of the results, comparing it to the ground truth (which we will explain the dataset and the input-ground truth pairs later).

3.2.1 Syntactic Similarity

Syntactic similarity measures whether or not two EREs are exactly the same. In other words, if you treated them as a string, would they be equivalent.

3.2.2 Semantic Similarity

Semantic similarity measures whether two different ERE accept the exact same language and evaluate traces identically (for every single existing trace, would ERE A and ERE B have the same result). For example, the expressions "eA eB eB*" and "eA eB+" are syntactically different but semantically similar (where eA and eB are events A and events B).

3.3 Related Work & Novelty

There exists prior work in determining whether or not LLMs can translate NL into formal logic. One such framework is nl2spec, which utilizes Codex, the

model fine-tuned on web crawled GitHub repositories, to achieve a 58% semantic accuracy in translating natural language to LTL. Some issue with this tool is that Codex is fine-tuned on general GitHub code, not formal logic. Further, the testing failed to have a reliable way to validate generated LTL, so the developers only tested on 20 NL-LTL pairs, opting for human expert validation [4]. Another, more recent, tool is Req2LTL, which utilizes an intermediate representation the researchers developed called OnionL. After translation NL to OnionL using transformer-based methods, they have a rule-based method which transforms OnionL to LTL. Their best model (GPT 4o) received a high 88.7% semantic accuracy. However, Req2LTL is specifically made for aerospace specifications, and not for runtime-verification specs [9]. Neither of these works extend well onto generating ERE for MOP.

For evaluation, we will utilize a dataset titled property-db. It contains 137 MOP files that each are paired with an NL that explains what the formal logic does [8]. Further, for fine-tuning, we will use a dataset of 55,000 RE that are paired with examples of matches and fails [1].

Through our methods, we aim to improve accuracy, test on larger datasets by developing a automated ERE comparison tool, and open-source the results (as neither Req2LTL nor nl2spec released their tool for public use).

4 Methods

4.1 Stakeholders and Success Criteria

Software engineers, quality assurance testers, and students who are dealing with large software systems, all who could be using RV, are stakeholders. Our success criteria is creating an accessible tool that lowers the barrier-of-entry for RV, generating ERE from NL. For success, we aim to have the generated ERE be at least valid ERE syntax over 90% of the time and we hope to receive a semantic accuracy of over 60%.

4.2 Semantic Comparison Engine

We develop a semantic comparison engine for ERE, building off of existing ideas of comparing RE. There are a few notable differences of ERE from RE as mentioned prior, being events instead of characters, creation events, and terminating events.

The engine operates through a sequential, multi-step algorithmic pipeline:

4.2.1 Event Anonymization

Standard finite state machine libraries are typically designed to process single-character alphabets rather than verbose event names like `use_file` or `modify_file`. The engine first extracts all unique events from both the generated and ground-truth EREs and maps them to a shared dictionary of single characters (e.g.,

A, B, C). Epsilon events (ϵ), which indicate an empty trace, are also normalized. This ensures the expressions are uniformly formatted for algorithmic evaluation.

4.2.2 Creation Events

Execution traces theoretically grow indefinitely, but a MOP monitor only begins live-tracking when an explicitly defined "creation" event occurs. To model this, the engine injects a novel gateway state at the start of the min-DFA. This state serves as a mandatory prerequisite, connecting to the initial events via the creation event's transition. This ensures expressions like A^* are correctly processed as requiring at least one A (A^+).

4.2.3 Terminating Events

If a MOP logic expression triggers a matching handler (@match), the monitor activates the moment the pattern is fulfilled; any subsequent events in the trace are irrelevant. To account for this, the engine iterates through all accepting (terminal) states in the min-DFA and programmatically deletes their outbound transition edges. Following this truncation, a root-node graph search is executed to identify and permanently delete any states or transitions that have been rendered unreachable.

4.2.4 Equivalence Verification

Once we have fixed all previous issues and turned it into a min-DFA, there exist well-known methods to compare two min-DFA to see if they are equivalent [5, 10].

4.3 Iterative Prompt Engineering & Fine-Tuning

We prompt engineer to give the model better context. Though LLMs like GPT-OSS, GPT 5.5, and GPT-4o do know what ERE, MOP, and RV is, it is still important to provide a dictionary, examples, etc. The generation process was an iterative feedback loop: we supplied an initial, very simple prompt to the OpenAI API, tested it on a small-scale preliminary test set, analyzed the remaining issues by-hand, revised the prompt instructions accordingly, then repeated the process until we reached a local maximum of accuracy on the preliminary set. Further, we utilized ideas from the OpenAI prompt engineering guide [11]. Figure 1 provides a visual for the iterative process we used.

This methodology integrated general principles from OpenAI's official prompt engineering guidelines and extended insights from B. Çakar's studies on standard regular expression generation into the Extended Regular Expression (ERE) domain.

Through six major iterations (Iteration 0 to Iteration 5), the prompt evolved from barebones to a prompt containing a lot of situational context and domain knowledge about generating ERE:

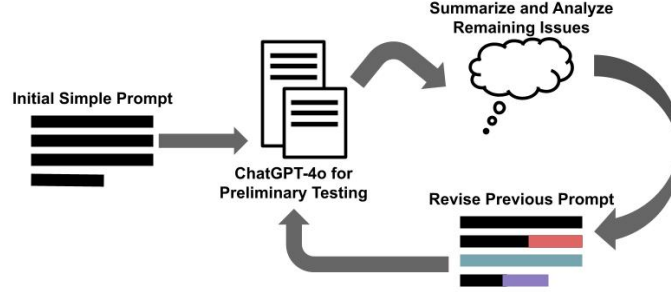


Figure 1: Flowchart showing how the prompt was iteratively refined. The model in the sample is GPT-4o but the idea can be replicated for all the other models we tested.

Table 1: Iterative Prompt Engineering Refinements and Performance

Modifications	Iteration	Prompt Addition/Change
	Iteration #0 (Accuracy 0/3)	Translate the following natural language into regular expression for java runtime-verification

Table 1 – continued from previous page

Modifications	Iteration	Prompt Addition/Change
Defines @fail and @match handlers. Provides ERE syntax dictionary. Elaborates more on characteristics of MOP (only events defined will be considered in the trace).	Iteration #1 (Accuracy 0/2)	The result of the regular expression will fall into two categories: match, meaning the expression matches the entire relevant trace, or fail, meaning the expression does not match onto the entire relevant trace. It should be indicated for the final expression which one of these results maps to a violation of the runtime property. Recall that regular expression uses "(" and ")" to group events, " " to signify "or", "*" to signify match as many as possible, and "+" to signify match as many as possible but at least once. For example, the expression "(A B)+" means match either event A or event B as many times as possible but at least once. In the regular expression, do not pass arguments to the events and only put the event name. Further, the expression will ignore any event not defined, so using "." is unnecessary.
Further elaborated on "only events defined will be considered in the trace." Elaborated on the logic that comes with "creation" events.	Iteration #2 (Accuracy 1/3)	Any events not named in the regular expression are automatically ignored and need not be accounted for using .*, other*, or any wildcard inside of the expression. The created regular expression should be as specific as possible and should begin from the creation of the object(s) in question.
Discourage usage of the ? operator and instead using epsilon.	Iteration #3 (Accuracy 1/3)	On the otherhand, operators like "?" and "{a,b}" do not exist and should not be used. For example, the expression "(A B)+" means match either event A or event B as many times as possible but at least once. Furthermore, this special form of Regex has added an event called "epsilon", which represents the event of nothing. For example, the expression "(A epsilon)" would match both the trace "A" and an empty trace.

Table 1 – continued from previous page

Modifications	Iteration	Prompt Addition/Change
Explicitly stated that all interactions should be considered in the ERE (A before B, B before A).	Iteration #4 (Accuracy 2/3)	To be able to match the relevant trace, analyze how the events in the natural language interact with each other at all times from creation. For example, even if the natural language specifies that event A cannot occur after event B, make sure to consider event A occurring before event B in order to consider the entire trace.
Explicitly stated that the trace can be unconventional, for example “shutdown” can be in the trace multiple times.	Iteration #5 (Accuracy 1/3)	In the trace, unconventional sequences events may occur, for example events like “terminate” and “start” could be ran multiple times. If the following natural language does not specify against it, cases like these need to be considered.

In Table 1, we notice some of the main issues that arise. First, LLM very often uses incorrect syntax that does not match ERE at all. Besides that, the next greatest issue is logic. If we allow the LLM to explain the logic, it often misses some cases (unconventional, but possible traces).

4.4 Natural Language and Code Clarification

To further improve accuracy, we are motivated to create the NL Clarifier and the Event Interpreter.

4.4.1 NL Clarifier

In both real-world settings and inside the property-db dataset containing Java API Javadocs, the NL requirements are often terse and assume knowledge the reader has (or assume the ability of the reader to explore more on their own). For example, a dataset property might simply state:

Prevents invocations of mark() if the Reader does not support.

Compared to the highly structured, mathematically precise prompts utilized in other related temporal logic frameworks, this ambiguous phrasing forces the generative model to guess the missing operational parameters. We will not open access to internet search for the model as that increases cost and reduces the speed of the tool too significantly.

To address this, the pipeline utilizes the NL Clarifier transformer model to expand on the input. We feed in the raw Java documentation, ask for the NL Clarifier to tie together the NL to the specific text from the primary source that

the Java docs are, and to clarify any potential misunderstandings. For example, the previous NL about invocations of `mark()` is expanded to the following:

Context:

Mark Supported: Invocation of `Reader.markSupported()` that evaluates to false. Mark Used: Invocation of `Reader.mark(int readAheadLimit)`.

Structured Rule:

IF Mark Unsupported is true for a given `Reader` instance,
THEN Mark Used MUST NOT occur on that instance.

`java.io.Reader.markSupported()`: "Tells whether this stream supports the `mark()` operation. The default implementation always returns false."

`java.io.Reader.mark(int)`: "Not all character-input streams support the `mark()` operation. [...] Throws: `IOException` - If the stream does not support `mark()`, or if some other I/O error occurs."

4.4.2 Event Interpreter

We are motivated to turn the events into NL since the model is far better at understanding NL. Especially as these MOP files increase in size, context proves to be a real issue in the understanding of models. Thus, we convert the events into NL. An example is as follows:

```
call(void Iterator+.remove()) & target(i)
```

Will be transformed into:

Remove: The iterator `i` has `remove()` called.

4.5 Fine-Tuning

We cannot fine-tune GPT 5.5 or any of the flagship models that can only be ran on compute provided by OpenAI. However, we can attempt to fine-tune GPT-OSS-120b, an open source model released by OpenAI [12]. We utilize Regex-CompBench, which contains about 55,000 real-world RE composition tasks, as the training data [1]. We utilized Kiln to be the tool which performed LoRA, or Low-Rank Adaptation, a fine-tuning method [3].

4.6 Final Program Structure

Figure 2 contains the diagram of the final architecture:

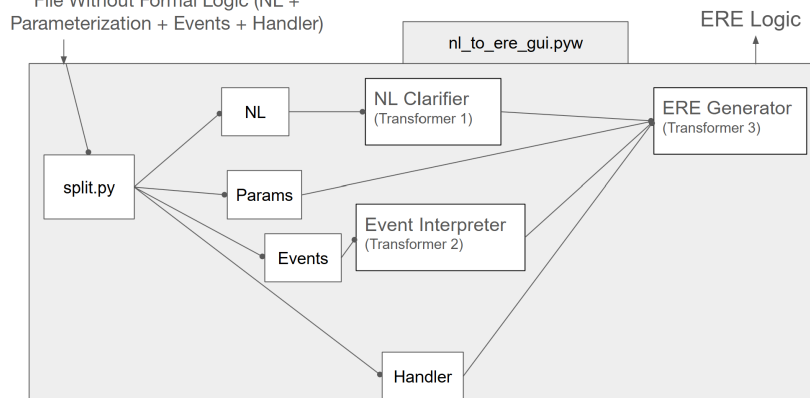


Figure 2: Flowchart showing how NL, Parameters, Events, and the Handler are passed into the tool to generate the final ERE

5 Results and Discussion

5.1 Test Data

Once again, the dataset which we used was property-db [8]. It contains 137 NL-MOP file pairs. To test, we downloaded every MOP file, then deleted the row which contained the ERE. Then, we passed the file path into the tool. After that, we utilized the ERE comparison tool to compare the generated ERE with the ground truth ERE.

5.2 Testing Structure

Figure 3 is a diagram of the testing architecture:

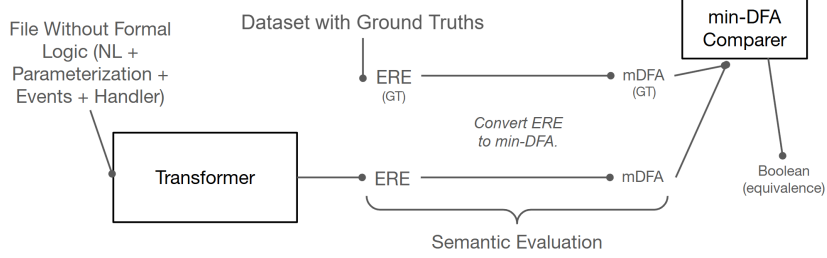


Figure 3: Flowchart showing how generated ERE is tested for semantic similarity

5.3 Testing and Results

Table 2: Performance of Evaluated Models on ERE Generation

Model Configuration	Semantic	Syntactic
GPT 5.5 w/ Iterative Prompt	0.42	0.32
GPT 5.5 w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.61	0.44
GPT-OSS w/ Iterative Prompt	0.22	0.15
GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.28	0.19
Fine-tuned GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.35	0.21

Table 2 contains the results to the testing of multiple strategies.

A total of 3 errors occurred while testing all 137 pairs, meaning 3 generated ERE were in invalid syntax (being about 2.2%).

We discuss the implications and summary of these results in the following section.

6 Conclusion

6.1 Summary of Findings

Our research does not boast the most attractive accuracy (61% at best) but it does show that the automated generation of ERE for RV is viable. Besides that, two other insights that we have shown is that clarification of the NL and

turning the Events to NL does provide a measurable improvement in accuracy (in terms of GPT 5.5, the difference was 65% vs 42%)

GPT 5.5 did better than the fine-tuned OSS model, to the point where GPT 5.5 without NL Clarifier nor Event Interpreter still outperformed the fine-tuned OSS model with both (42% vs 35%).

Finally, the existence of an ERE comparator tool did greatly improve the strength and reliability of our results, as we were able to test on a much larger dataset than some previous work. Further, We are able to see that if we had solely compared syntactically, the results would have been understated (with 0.61% semantic vs 0.44% syntactic accuracy for GPT 5.5 with all addons).

6.2 Project Impact

The development of the multi-transformer architecture serves to be an important base of creating a more sophisticated model. 61% accuracy is a step up from tools like nl2spec, and shows that it is possible for better methods to further improve upon this idea of automatically generating ERE for RE. At this time, 61% accuracy is still too low to use reliably, so we do not suggest using it as a replacement for writing ERE, but instead as being able to potentially create an initial ERE that developers can build off of.

Further, we open-source the entire project, with instructions to use both the comparator tool and the translation tool inside the project Github [6].

6.3 Future Work

We believe we should conduct a wide survey of LLMs, as this project only tested the capabilities of models in the OpenAI ecosystem.

We also believe it could be worthwhile to explore whether different formal logic languages (such as LTL) are more easily generated by the models. Perhaps switching to LTL and attempting to perform this project again could work better.

Lastly, we believe that we should attempt to test the tool in a real-world industry setting, trying to generate ERE for production sized software-systems. This would allow us to get a better sense of how well the tool does if used by real software-developers.

References

- [1] B. Çakar et al. “Is Reuse All You Need? A Systematic Comparison of Regular Expression Composition Strategies”. In: *arXiv preprint arXiv:2503.20579* (2025). URL: <https://arxiv.org/abs/2503.20579>.
- [2] Feng Chen and Grigore Roşu. “Mop: an efficient and generic runtime verification framework”. In: *SIGPLAN Not.* 42.10 (2007), pp. 569–588. DOI: 10.1145/1297105.1297069.
- [3] Chesterfield Laboratories Inc. *Kiln: Rapid AI Prototyping and Dataset Collaboration Tool*. Version latest. 2025. URL: <https://github.com/Kiln-AI/Kiln>.
- [4] M. Cosler et al. “nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models”. In: *arXiv preprint arXiv:2303.04864* (2023). URL: <https://arxiv.org/abs/2303.04864> (visited on 11/07/2025).
- [5] CyberZHG. *Toolbox*. 2023. URL: <https://cyberzhg.github.io/toolbox/>.
- [6] D. Feng. *Generating-ERE-from-NL-SRP-Yilmaz-7*. Version latest. 2026. URL: <https://github.com/dxfeng/Generating-ERE-from-NL-SRP-Yilmaz-7>.
- [7] D. Jin et al. “JavaMOP: Efficient parametric runtime monitoring framework”. In: *35th International Conference on Software Engineering (ICSE)*. 2012, pp. 1427–1430. DOI: 10.1109/icse.2012.6227231.
- [8] O. Legunsun. *property-db*. 2025. URL: <https://github.com/SoftEngResearch/tracemop/tree/3de76a81606fce70e893ab305320b310a614438a/scripts/props>.
- [9] Z. Ma et al. “Bridging Natural Language and Formal Specification—Automated Translation of Software Requirements to LTL via Hierarchical Semantics Decomposition Using LLMs”. In: *40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Seoul, Korea, Republic of, 2025, pp. 1208–1220. DOI: 10.1109/ASE63991.2025.00104.
- [10] Daphne Norton. “Algorithms for testing equivalence of finite automata, with a grading tool for JFLAP”. MA thesis. Rochester Institute of Technology, 2009. URL: <https://repository.rit.edu/cgi/viewcontent.cgi?article=7944&context=theses>.
- [11] OpenAI. *Prompt engineering*. 2026. URL: <https://platform.openai.com/docs/guides/prompt-engineering>.
- [12] OpenAI et al. *gpt-oss-120b gpt-oss-20b Model Card*. 2025. arXiv: 2508.10925 [cs.CL]. URL: <https://arxiv.org/abs/2508.10925>.